

SPRAWOZDANIE

Zajęcia: Grafika komputerowa
Prowadzący: mgr inż. Jakub Krawczyk

Laboratorium: 7

Temat: "WebGL"

Wariant: 1

Jakub Kleczek
Informatyka I stopień,
stacjonarne,
4 semestr,
Gr.1b

1. Polecenie

Program w **lab11.html** pokazuje wiele ruchomych czerwonych kwadratów, które odbijają się od krawędzi płótna. Płótno wypełnia cały obszar zawartości przeglądarki internetowej. Kwadraty odpowiadają również myszy: Jeśli klikniesz lewym przyciskiem myszy lub klikniesz lewym przyciskiem myszy i przeciągniesz na płótnie, cały kwadrat będzie kierowany w stronę pozycji myszy. Jeśli klikniesz lewym przyciskiem myszy, dane punktów zostaną ponownie zainicjowane, więc zaczną się od środka. Możesz wstrzymać i ponownie uruchomić animację, naciskając spację.

Kwadraty są w rzeczywistości częścią jednego prymitywu WebGL typu `gl.POINTS`. Każdy kwadrat odpowiada jednemu z wierzchołków pierwotnego. Oczywiście renderowanie jest wykonywane przez moduł shadera wierzchołków i moduł shadera fragmentu. Kod źródłowy shaderów jest w dwóch fałszywych „skryptach” w górnej części pliku html.

Będziesz modyfikował kod modułu shadera i kod JavaScript, aby zaimplementować kilka różnych stylów dla prymitywu punktu. Na przykład możliwe będzie rysowanie kwadratów w różnych kolorach, rysowanie wielokątów zamiast kwadratów i tak dalej. Użytkownik będzie kontrolował program, naciskając klawisze na klawiaturze. Do ciebie należy decyzja, których klawiszy użyć, ale proszę udokumentować interfejs w odpowiednim komentarzu do funkcji `doKey()` lub na górze programu.

Program ma dwie funkcje, nad którymi będziesz musiał pracować: Funkcja `initGL()` jest wywoływana, gdy program jest uruchamiany po raz pierwszy, a funkcje `updateForFrame()` i `render()` są wywoływane dla każdej ramki animacji. Ten sam zestaw poleceń byłby legalny we wszystkich tych poleceniach, ale `initGL()` jest najlepszym miejscem do ustawiania rzeczy, które nie zmieniają się w trakcie działania programu, takich jak położenie zmiennych i zmiennych atrybutów w module shadera; `updateForFrame()` jest przeznaczony do aktualizacji zmiennych JavaScript, które zmieniają się z ramki na ramkę; i `render()` ma na celu wykonanie rzeczywistego rysunku WebGL ramki.

Atrybut koloru

W oryginalnej wersji programu wszystkie kwadraty są czerwone. Pierwsze ćwiczenie polega na umożliwieniu przypisania innego koloru do każdego kwadratu. Ponieważ kwadraty są naprawdę wierzchołkami w pojedynczym prymitywie typu `gl.POINTS`, można użyć zmiennej atrybutu dla koloru. Atrybut może mieć inną wartość dla każdego wierzchołka.

Pierwszym zadaniem jest dodanie zmiennej kolorowej typu `vec3` do modułu shadera wierzchołka i użycie wartości atrybutu do pokolorowania kwadratów. Będziesz także musiał pracować po stronie JavaScript. Będziesz potrzebował `Float32Array` do przechowywania wartości kolorów po stronie JavaScript, a będziesz potrzebował bufora WebGL dla tego atrybutu. Program ma już jeden atrybut, który jest używany do współrzędnych wierzchołków. Będziesz robił coś podobnego do atrybutu `color` (poza tym, że możesz to zrobić w `initGL()`, ponieważ wartości kolorów nie zmieniają się po ich utworzeniu). Można użyć losowych wartości w zakresie od 0,0 do 1,0 dla składników koloru.

Po uruchomieniu wielokolorowych kwadratów powinieneś ustawić kolory jako opcjonalne. Możesz włączać i wyłączać użycie tablicy wartości atrybutów za pomocą następujących poleceń, gdzie `a_color_loc` to identyfikator atrybutu `color` w programie shader:

```
gl.enableVertexAttribArray (a_color_loc); // użyj bufora atrybutów kolorów  
gl.disableVertexAttribArray (a_color_loc); // nie używaj bufora
```

Gdy tablica atrybutów jest włączona, każdy wierzchołek otrzymuje swój własny kolor z buforu atrybutów. Gdy tablica atrybutów jest wyłączona, wszystkie wierzchołki otrzymują ten sam kolor, a tę wartość można ustawić za pomocą rodziny funkcji `gl.vertexAttrib *`. Na przykład, aby ustawić wartość używaną, gdy tablica atrybutów kolorów jest wyłączona, można użyć

```
gl.vertexAttrib3f (a_color_loc, 1, 0, 0); // ustaw kolor attribute na czerwony
```

Pozwól użytkownikowi na naciśnięcie określonego klawisza, aby włączyć lub wyłączyć losowe kolory. Program ma funkcję `doKey()`, która jest już skonfigurowana do reagowania na wprowadzanie z klawiatury. Będziesz dodawać do programu kilka typów interakcji z klawiaturą. Aby odpowiedzieć na klawisz, musisz znać numeryczny kod klawiszy. Funkcja `doKey()` wysyła kod do konsoli za każdym razem, gdy użytkownik uderza klawisz, i możesz użyć tej funkcji, aby odkryć wszystkie inne kody klawiszy, których potrzebujesz.

Styl punktów

Powinieneś dodać opcję używania stylu wyświetlania dla punktów w postaci wielokąta. Pozwól użytkownikowi wybrać styl za pomocą klawiatury; na przykład, naciskając klawisze numeryczne.

Style będą musiały zostać zaimplementowane w shaderze fragmentu, a będziesz potrzebował nowej zmiennej jednolitej, aby powiedzieć modułowi shadera fragmentu, którego stylu użyć. Dodaj jednolitą zmienną typu `int` do shadera fragmentu, aby kontrolować styl punktu, i dodaj kod do modułu cieniującego fragmentu, aby zaimplementować różne style. Będziesz także musiał dodać zmienną po stronie JavaScript dla lokalizacji zmiennej jednolitej, a będziesz musiał wywołać `glUniform1i`, gdy chcesz zmienić styl.

Naprzykład, żeby narysować punkt jako dysk, odrzucając niektóre piksele:

```
float dist = distance( vec2(0.5), gl_PointCoord );  
if (dist > 0.5) {  
    discard;  
}
```

Powinieneś również wykorzystać przezroczystość alfa w niektórych stylach. Aby umożliwić korzystanie ze składnika `alpha`, musisz dodać następujące linie do funkcji `initGL ()`:

```
gl.enable(gl.BLEND);  
gl.blendFunc(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);
```

Dzięki tym ustawieniom wartość alfa koloru będzie używana do przezroczystości w zwykły sposób. W szczególności jeden z twoich stylów powinien pokazywać punkt jako wielokąt, który zanika z całkowicie nieprzezroczystego w środku wielokątu do całkowicie przezroczystego na krawędzi.

2. Wprowadzone dane (fragmenty)

```
function render() {
  gl.clear(gl.COLOR_BUFFER_BIT);

  gl.bindBuffer(gl.ARRAY_BUFFER, a_coords_buffer);
  gl.bufferData(gl.ARRAY_BUFFER, positions, gl.STREAM_DRAW);
  gl.vertexAttribPointer(a_coords_loc, 2, gl.FLOAT, false, 0, 0);

  if (isColorRandom) {
    gl.bindBuffer(gl.ARRAY_BUFFER, a_color_buffer);
    gl.vertexAttribPointer(a_color_loc, 3, gl.FLOAT, false, 0, 0);
    gl.enableVertexAttribArray(a_color_loc);
  }
  else {
    gl.disableVertexAttribArray(a_color_loc);
    gl.vertexAttrib3f(a_color_loc, 1, 0, 0);
  }

  gl.uniform1i(u_style_loc, pointStyle);
  gl.drawArrays(gl.POINTS, 0, POINT_COUNT);

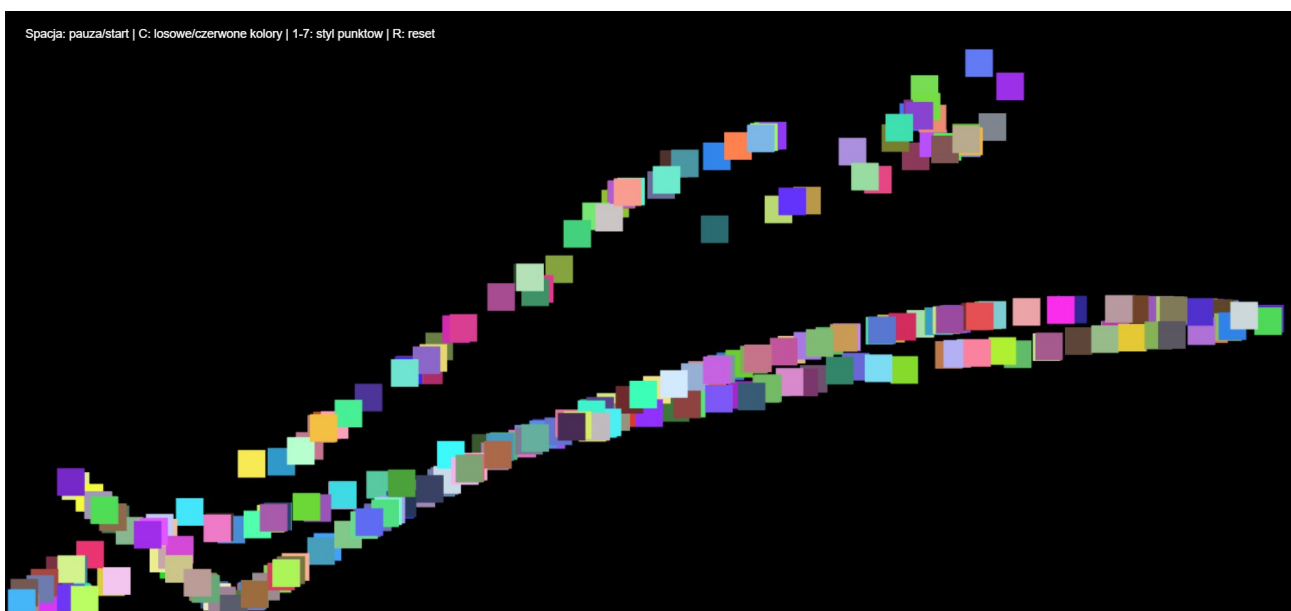
  if (gl.getError() !== gl.NO_ERROR) {
    console.log("During render, a GL error has been detected.");
  }
}
```

```
function randomizeColors() {
  for (var i = 0; i < POINT_COUNT; i++) {
    colors[3 * i] = 0.15 + 0.85 * Math.random();
    colors[3 * i + 1] = 0.15 + 0.85 * Math.random();
    colors[3 * i + 2] = 0.15 + 0.85 * Math.random();
  }
}
```

```
float polygonMask(float sides, float radius, vec2 p) {
  float angle = atan(p.y, p.x) + PI;
  float sector = 2.0 * PI / sides;
  float local = mod(angle + sector * 0.5, sector) - sector * 0.5;
  return cos(local) * length(p) <= radius ? 1.0 : 0.0;
}

float polygonDistance(float sides, vec2 p) {
  float angle = atan(p.y, p.x) + PI;
  float sector = 2.0 * PI / sides;
  float local = mod(angle + sector * 0.5, sector) - sector * 0.5;
  return cos(local) * length(p);
}
```

3. Wynik działania



4.Wykorzystane komendy:

[https://github.com/DziadekSwinka/Grafika/tree/](https://github.com/DziadekSwinka/Grafika/tree/872ba7d58c9a4b67a3925f2e21622b65b0abe453/Lab7%20-%20WEBGL)

[872ba7d58c9a4b67a3925f2e21622b65b0abe453/Lab7%20-%20WEBGL](https://github.com/DziadekSwinka/Grafika/tree/872ba7d58c9a4b67a3925f2e21622b65b0abe453/Lab7%20-%20WEBGL)